

SLIDE 1 — WHAT IS THE DEREFERENCE OPERATOR?

MEET THE DEREFERENCE OPERATOR `*`!

When working with **pointers**, a variable can store the **address** of another variable.

The dereference operator `*` is used to **access the value stored at that address**.

Simple Definition

Dereferencing means using `*` to access the value stored at a memory address.

LOOK AT THIS!

None

```
int x = 10;  
int *p = &x;
```

Here:

None

```
x → 10  
p → address of x
```

When we use:

None

```
*p
```

we get:

None

```
*p → 10
```

So:

p → **Address** 
***p** → **Value** 

EASY ANALOGY

Think of **p** as a **house address** 

None

p → House Address 
***p** → What's inside the house 

The address tells us **where to go**.

The ***** lets us access **what is stored there**.

★ REMEMBER!

p → **ADDRESS** 
***p** → **VALUE** 

SLIDE 2 — ACCESSING THE VALUE USING *

HOW DOES DEREFERENCING WORK?

First, we create a normal variable:

None

```
int x = 25;
```

Then create a pointer:

None

```
int *p = &x;
```

Now:

None

```
x → 25
```

```
p → address of x
```

To access the value through the pointer:

None

```
*p
```

So:

None

```
*p → 25
```

LOOK AT THIS!

None

```
int x = 25;
```

```
int *p = &x;
```

```
printf("%d", *p);
```

OUTPUT:

None

```
25
```

EASY ANALOGY

Think of:

None

p → 📌 Location
*p → 📦 Open the location

The pointer tells us **where the value is**.

The * tells C:

“Go to that address and give me the value stored there.”

★ REMEMBER!

&x → GET ADDRESS 📌

p → STORE ADDRESS 📌

*p → GET VALUE 🔍

🟡 SLIDE 3 — 🖋️ CHANGING A VALUE USING *

😞 CAN WE CHANGE THE VALUE USING A POINTER?

Yes! 🎯

We can use the dereference operator to **change the value stored at the address**.

📄 LOOK AT THIS!

None

```
int x = 10;  
int *p = &x;  
*p = 20;
```

What happens?

Before:

None

$x = 10$

After:

None

$*p = 20$

Now:

None

$x = 20$

Because p points to x .

VISUALIZE IT

None

p

↓



x

|

└─

10

After:

None

$*p = 20;$

None

p

↓



x

|

└─

20

EASY ANALOGY

Think of `p` as a key  to a locker.

None

```
p    → Finds the locker   
*p   → Opens the locker  
*p = 20 → Changes what's inside
```

BIG MEMORY TRICK

```
*p → ACCESS VALUE   
*p = new_value → CHANGE VALUE 
```

★ TAKEAWAY

A pointer can be used to **read and modify** the value stored at its address.

SLIDE 4 — USING THE DEREFERENCE OPERATOR IN C

CAN WE USE `*` WITH `printf`?

Yes! 

None

```
int x = 50;  
int *p = &x;  
printf("%d", *p);
```

OUTPUT:

None

50

Because:

None

`x → 50`

`p → address of x`

`*p → 50`

🤔 **WHAT IF WE CHANGE `*p`?**

None

```
int x = 50;
```

```
int *p = &x;
```

```
*p = 100;
```

```
printf("%d", x);
```

OUTPUT:

None

`100`

Why?

Because `p` points to `x`.

So:

None

```
*p = 100;
```

changes the value of `x`.

🧠 **BIG MEMORY TRICK**

None

`& → GET ADDRESS` 📌

`p → STORE ADDRESS` 📌

* → GET VALUE 🔍

*p = value → CHANGE VALUE ✎

★ TAKEAWAY

* lets us access the value stored at the address held by a pointer.

📌 SLIDE 5 — 🏆 DEREFERENCE OPERATOR RECAP 🔄

🚀 WHAT DID WE LEARN?

📌 1. GET THE ADDRESS

None

```
p = &x;
```

& → Gets the address

🔗 2. POINTER STORES THE ADDRESS

None

```
int *p = &x;
```

p → Stores the address of x

🔍 3. DEREFERENCE THE POINTER

None

```
*p
```

***p** → Gets the value stored at that address

4. CHANGE THE VALUE

None

```
*p = 20;
```

Changes the value of the variable that **p** points to.

ONE BIG MEMORY TRICK!

None

&x

↓

GET ADDRESS 

↓

p

↓

STORES ADDRESS 

↓

***p**

↓

GET VALUE 

↓

***p = 20**

↓

CHANGE VALUE 

 **REMEMBER!**

& → ADDRESS 
p → POINTER 
*p → VALUE 



QUESTIONS

1 What does *p do?

- A) Gets the address of p
- B) Gets the value stored at the address in p
- C) Creates a new variable
- D) Deletes the pointer

 **Hint:** Dereferencing is about accessing the **value**.

Glossary:

Dereference → Access the value stored at the address held by a pointer.

2 What is the output?

None

```
int x = 25;  
int *p = &x;  
printf("%d", *p);
```

- A) Address of x
- B) 25
- C) 0
- D) p

 **Hint:** *p accesses the value stored at the address.

Glossary:

Value → The data stored inside a variable.

3 What is the value of x?

None

```
int x = 10;
```

```
int *p = &x;  
*p = 30;
```

- A) 10
- B) 20
- C) 30
- D) Address of `x`

 **Hint:** `p` points to `x`.

 **Glossary:**

Points to → A pointer stores the address of a variable.

4 What is the output?

None

```
int x = 40;  
int *p = &x;  
*p = 60;  
printf("%d", x);
```

- A) 40
- B) 60
- C) Address of `x`
- D) 0

 **Hint:** Changing `*p` changes the variable pointed to by `p`.

 **Glossary:**

Pointer → Stores the address of another variable.

5 Which statement changes the value of `x` to 75?

None

```
int x = 10;  
int *p = &x;
```

- A) `p = 75;`
- B) `&p = 75;`

C) `*p = 75;`

D) `x * 75;`



Hint: Use dereferencing to modify the value.



Glossary:

Modify → Change an existing value